Part 1 Introduction

In this lab, we integrate cache memory systems into a pipelined RISC-V processor to improve data access performance and gain practical experience with memory hierarchy design. We implement a direct-mapped instruction cache (I-cache) and a two-way set-associative data cache (D-cache), using RAM modules to construct separate tag and data arrays, along with valid, dirty, and LRU state management. Additionally, a victim cache is incorporated to reduce conflict misses and enhance performance. After verifying correctness through comprehensive directed and random tests, we evaluate overall system efficiency under different cache configurations (None, Icache, Dcache, All) using various benchmarks with variable memory latency. The results highlight performance differences and design trade-offs across configurations, demonstrating the impact of cache architecture—especially the victim cache—on improving processor performance.

Part 2 Design

I-cache:

- Specifications: 2KB, direct mapping, 64B block, write allocation.
- Parallel tag/data reading to support continuous hits without bubbles.
- Read the data returned within 1-cycle of the hit.
- In write operations, only write-allocate and update data; dirty bits are not processed.

D-cache:

- Specifications: 4KB, 2-way set associative, 64B block.
- Maintenance per group: Valid bit / dirty bit and LRU bit (1 bit is enough to record less frequently used ways)
- Hit: Determined by comparing the result with the tag and valid bits.
- Miss: First check Victim Cache (if a hit occurs, swap). Second eliminate the block in the primary cache according to the LRU decision. If dirty, then spill-before-fill.

Victim-cache:

- 2-entry fully associative.
- Store blocks that have been evicted from the main cache.
- Behavioral Flow: D-cache miss → Check Victim Cache.If Victim hit → Cache swap.If Victim is full and dirty → Prioritize write-back before replacement.

Control Flow / FSM Adjustment:

To support continuous read hits without stalling .Design spill-before-fill and Victim Cache recovery processes and employ multi-stage pipelined write-back and refill data .Therefore, additional handshaking and conditional checks are added for states such as READ_CACHE, UPDATE, READ_MEM_REQ/RESP.

Discussion and what and why implementation:

In this cache design, I adopted several architectural decisions that balance performance and hardware feasibility. First, the tag array and data array are implemented separately using RAM-based structures, ensuring scalability and better alignment with realistic hardware behavior. For the data cache, a two-way set-associative organization with an LRU replacement policy is employed to effectively reduce conflict misses and improve the overall hit rate. A victim cache is additionally integrated to mitigate frequent conflict misses that typically occur in direct-mapped designs. To ensure data consistency, the spill-before-fill approach is applied whenever a dirty block is evicted, guaranteeing correctness during write-back operations. Finally, to meet the requirement of completing read hits in a single cycle, tag comparison and data access are performed in parallel to deliver high-performance cache responses.

Part 3 *Testing Methodology*

Tset1.S:

```
#=====================================================================
# 1 Clean line read/write hit
#=====================================================================
test_clean_hit:
        la   x10, data_region    # base
        li   x11, 0xAAAAAAA1
        sw   x11, 0(x10)         # write (miss + refill)
        lw   x12, 0(x10)         # hit
        mv   x21, x12
        mv   x20, x11
        jal  x0, check_equal
        ret
```

Test2.S

```
#=====================================================================
# 2 Dirty line read/write hit
#=====================================================================
test_dirty_hit:
        la   x10, data_region
        li   x11, 0xBBBBBBB2
        sw   x11, 4(x10)         # hit sets dirty
        lw   x12, 4(x10)
        mv   x21, x12
        mv   x20, x11
        jal  x0, check_equal
        ret
```

Test3.S

```
#=====================================================================
# 4 Miss + refill + eviction (conflict enforced)
#=====================================================================
test_refill_evict:
        la x10, data_region
        # force same index but different tags
        sw x0, 0(x10)            # fill set
        sw x0, 256(x10)          # conflict → eviction needed
        li x11, 0xDDDDDDD4
        sw x11, 512(x10)         # another conflict
        lw x12, 512(x10)
        mv x21, x12
        mv x20, x11
        jal x0, check_equal
        ret
```

Test4.S

```
#=====================================================================
# 4 Miss + refill + eviction (conflict enforced)
#=====================================================================
test_refill_evict:
        la x10, data_region
        # force same index but different tags
        sw x0, 0(x10)            # fill set
        sw x0, 256(x10)          # conflict → eviction needed
        li x11, 0xDDDDDDD4
        sw x11, 512(x10)         # another conflict
        lw x12, 512(x10)
        mv x21, x12
        mv x20, x11
        jal x0, check_equal
        ret
```

Test5.S

```
#=====================================================================
# 5 conflict miss stress (thrash same set)
#=====================================================================
test_conflict_same_set:
        la x10, data_region
        li x11, 0
loop_conflict:
        sw x11, 0(x10)
        lw x12, 256(x10)         # assume same index mapping
        addi x11, x11, 1
        li x13, 32
        blt x11, x13, loop_conflict
        ret
```

Test6.S

```
#=====================================================================
# 6 Capacity test → fill every line of cache
#=====================================================================
test_capacity_fill:
        la x10, data_region
        li x11, 0
cap_loop:
        sw x11, 0(x10)
        addi x10, x10, 32        # step 32B to walk new line
        addi x11, x11, 1
        li x13, 128              # match your cache total lines
        blt x11, x13, cap_loop
        ret
```

Tset7.S:

```
#==============================================================
# 7 LRU Test (for N-way set assoc)
#==============================================================
test_lru_policy:
        la x10, data_region
        sw x0, 0(x10)           # fill Way 0
        sw x0, 256(x10)         # fill Way 1
        sw x0, 512(x10)         # fill Way 2
        sw x0, 768(x10)         # fill Way 3
        lw x0, 0(x10)           # make Way0 MRU
        sw x0, 1024(x10)        # expect Way1 eviction if 4W
        ret
```

Test8.S

```
#==============================================================
# 8 Stride patterns
#==============================================================
test_stride_access:
        la x10, data_region
        li x11, 8
stride_loop:
        sw x11, 0(x10)
        lw x12, 0(x10)
        addi x11, x11, 1
        addi x10, x10, 128       # stride access
        li x13, 33
        blt x11, x13, stride_loop
        ret
```

Result

```
[ PASSED ] riscv-test1-bc.out
[ PASSED ] riscv-test2-bc.out
[ PASSED ] riscv-test3-bc.out
[ PASSED ] riscv-test4-bc.out
[ PASSED ] riscv-test5-bc.out
[ PASSED ] riscv-test6-bc.out
[ PASSED ] riscv-test7-bc.out
[ PASSED ] riscv-test8-bc.out
```

```
[ PASSED ] riscv-test1-bc-rand.out
[ PASSED ] riscv-test2-bc-rand.out
[ PASSED ] riscv-test3-bc-rand.out
[ PASSED ] riscv-test4-bc-rand.out
[ PASSED ] riscv-test5-bc-rand.out
[ PASSED ] riscv-test6-bc-rand.out
[ PASSED ] riscv-test7-bc-rand.out
[ PASSED ] riscv-test8-bc-rand.out
```

Conclusion

Through comprehensive cache functionality verification, we designed multiple directed test scenarios covering hit/miss behavior, dirty line write-back, replacement policies, capacity saturation, conflict accesses, and stride access patterns. All tests were executed under both fixed-latency and randomized-latency environments. Since every test case passed successfully, the results demonstrate that the implemented cache exhibits correct behavior in terms of data consistency, replacement logic, write-back mechanism, and timing robustness. These results confirm that the cache module can correctly handle diverse memory access situations that may occur in real execution, fully achieving the design and verification goals of Lab 5.

Part 4 *Evalution*

Benchmark (cycle count)	None	Icache	Dcache	All
Bin-search-rand	1101042	28381	145576	82026
cmplx-mult-rand	10114220	92022	173268	80934
masked-filter-rand	13450246	194634	482762	184336
vvadd-rand	3121351	27194	57588	40154

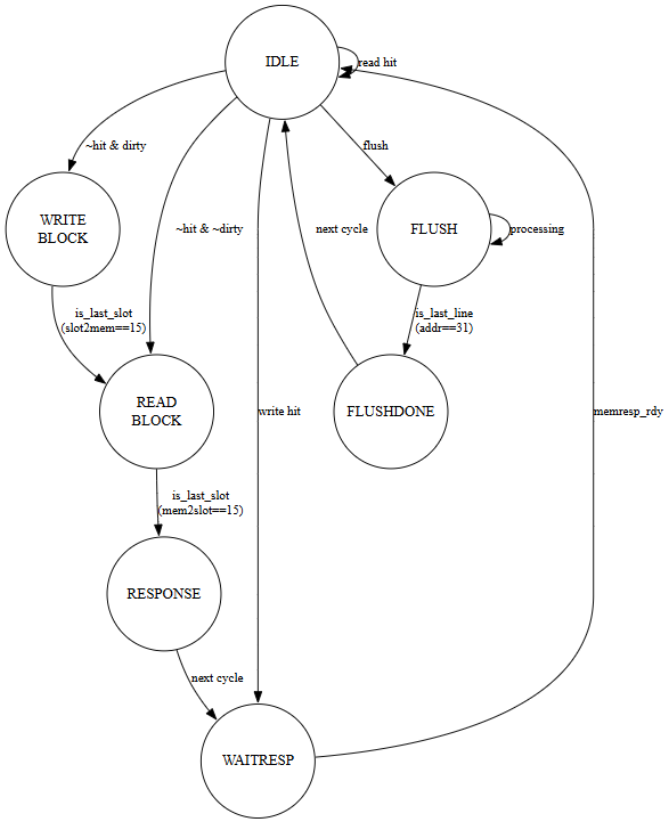
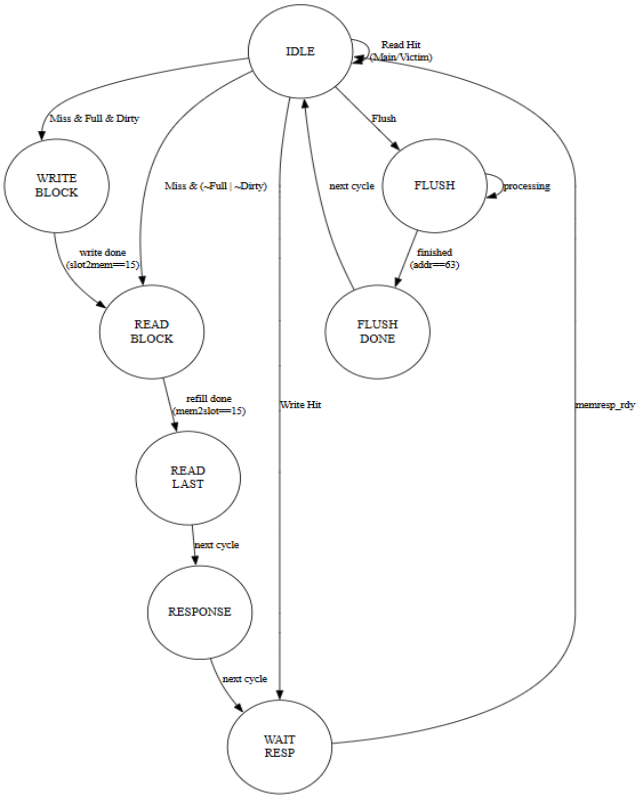
Benchmark (ipc)	None	Icache	Dcache	All
Bin-search-rand	0.000935	0.036292	0.007075	0.012557
cmplx-mult-rand	0.000170	0.018724	0.009944	0.021289
masked-filter-rand	0.000354	0.024477	0.009868	0.025844
vvadd-rand	0.000145	0.016768	0.007866	0.011356

Part 5 Discussion

The results show that enabling cache significantly improves performance in random-delay memory conditions. With Icache, instruction fetch stalls are reduced, but data accesses still frequently miss, leading to limited overall improvement. Dcache brings a much larger performance gain since it removes most memory stalls caused by load/store operations, which dominate the tested workloads. The All configuration achieves the best performance by accelerating both instruction and data paths simultaneously, minimizing pipeline stalls.Moreover, the Victim Cache in the Dcache design helps prevent conflict misses and reduces unnecessary write-backs to main memory. This further enhances performance—especially in random access patterns—making Dcache and All clearly superior to Icache alone.

Part 6 FIGURE

I-cagche FSM



D-cagche FSM